

第三章 回溯法

Chapter 3 Backtracking

计算机学院 潘永生

3.1 回溯法的基本思想

- 对于有些最优解问题，没有任何的理论也无法采用精确的数学公式来帮助我们找到最优解，我们只能用穷举算法。在这里我们介绍一种系统化的穷举搜索技术，称为回溯技术。
- 所谓回溯技术就是像人走迷宫一样，先选择一个前进方向尝试，一步步试探，在遇到死胡同不能再往前的时候就会退到上一个分支点，另选一个方向尝试，而在前进和回撤的路上都设置一些标记，以便能够正确返回，直到达到目标或者所有的可行方案都已经尝试完为止。

3.1 回溯法的基本思想

- 在通常的情况下，我们使用递归方式来实现回溯技术，也就是在每一个分叉点进行递归尝试。在回溯是通常采用栈来记录回溯过程，使用栈可使穷举过程能回溯到所要的位置，并继续在指定层次上往下穷举所有可能的解。
- 回溯法在问题的解空间树中，按深度优先策略，从根结点出发搜索解空间树。算法搜索至解空间树的任意一点时，先判断该结点是否包含问题的解。如果肯定不包含，则跳过对该结点为根的子树的搜索，逐层向其祖先结点回溯；否则，进入该子树，继续按深度优先策略搜索。

生成问题状态的基本方法

- 活结点:一个自身已生成但其儿子还没有全部生成的节点称做活结点
- 死结点:一个所有儿子已经产生的结点称做死结点
- 深度优先的问题状态生成法：如果对一个扩展结点R，一旦产生了它的一个儿子C，就把C当做新的扩展结点。在完成对子树C（以C为根的子树）的穷尽搜索之后，将R重新变成扩展结点，继续生成R的下一个儿子（如果存在）

生成问题状态的基本方法

- 回溯法：为了避免生成那些不可能产生最佳解的问题状态，要不断地利用限界函数(bounding function)来处死那些实际上不可能产生所需解的活结点，以减少问题的计算量。具有限界函数的深度优先生成法称为回溯法

3.1 回溯法的基本思想

回溯法对解空间作深度优先搜索，因此，在一般情况下用递归方法实现回溯法。

```
void backtrack (int t)
{
    if (t>n) output(x);
    else
        for (int i=f(n,t);i<=g(n,t);i++) {
                x[t]=h(i);
                if (constraint(t)&&bound(t))
                    backtrack(t+1);
            }
}
```

3.1 回溯法的基本思想

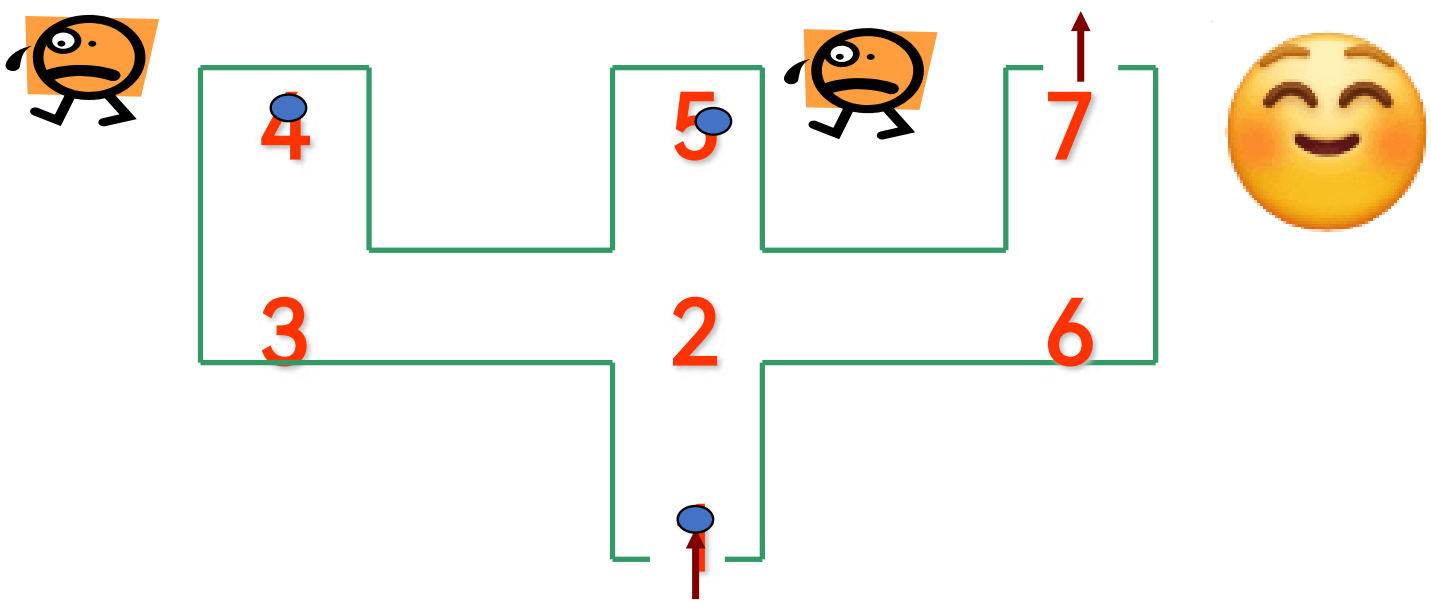
(1)针对所给问题，定义问题的解空间；
(2)确定易于搜索的解空间结构；
(3)以深度优先方式搜索解空间，并在搜索过程中用剪枝函数避免无效搜索。

常用剪枝函数：
用约束函数在扩展结点处剪去不满足约束的子树；
用限界函数剪去得不到最优解的子树。

回溯算法用伪代码描述如下：

```
Proc search(当前状态);
Begin
If 当前状态等于目标状态 then exit;
        for 对所有可能的新状态
                search(新状态);
End;
```

- Problem
  - A maze is a set of intersections。 The traveler enters from one direction and departs along one of three paths: to the left, straight ahead or to the right。 A path is identified by the number of the next intersection。



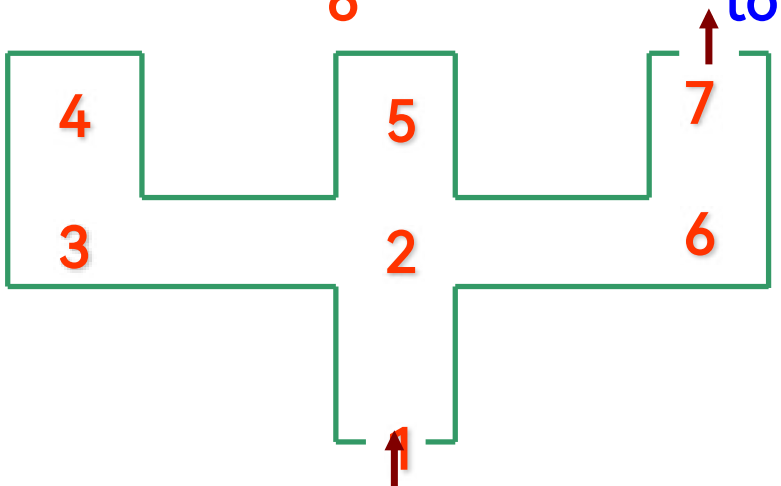
Demo

Backtracking

- Solution: backtracking
  - In an effort to obtain a final solution to the maze, we create a series of partial solutions step by step that appear to be consistent with the requirements of the final solution. If we take a step or make a decision that is inconsistent with a final solution, we backtrack one or more steps to the last consistent partial solution.

Mini Maze

Intersections		Action		Result
1	(Entrance)	straight	enter 2	
2		to left	enter 3	
3		to right	enter 4	
4	(Dead-end)	backtrack		enter 3
3	(Dead-end)	backtrack		enter 2
2		straight	enter 5	
5	(Dead-end)	backtrack		enter 2
2		to right	enter 6	
6		to left	enter 7	
			(end)	



Intersections structure

```
struct Intersection {
    int left;
    int forward;
    int right;
}
```

Data for mini Maze

```
6
0       2       0
3       5       6
0       0       4
0       0       0
0       0       0
7       0       0
7
```

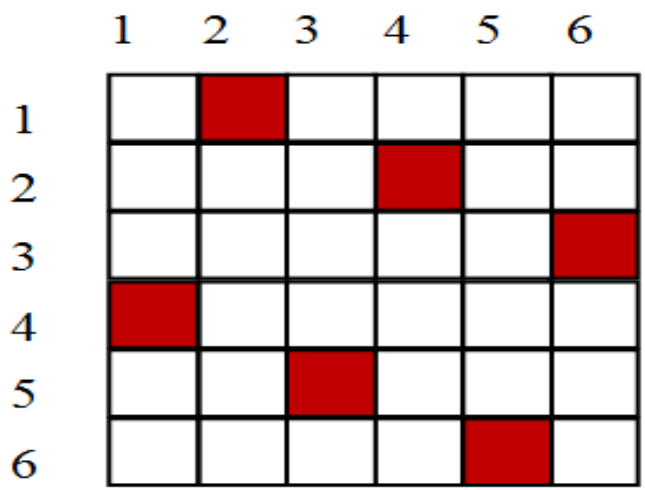
Maze Class

```
#include <iostream.h>
#include <fstream.h>
#include <stdlib.h>
class Maze {
private:
    int MazeSize;
    int EXIT;
    Intersection *intsec;
public:
    Maze ( char *filename );
    int TraverseMaze ( int
            CurrentPos );
}
```

Solution of the maze using backtracking

```
int Maze::TraverseMaze ( int CurrentPos ) {
  if ( CurrentPos > 0 ) {                    //from the intersection 1
    if ( CurrentPos == EXIT ) {              //stopping condition
      cout << CurrentPos << "   "; return 1;
    }
    else              //attemp to go left
    if (TraverseMaze(intsec[CurrentPos].left ))
      { cout << CurrentPos << "   "; return 1; }
    else              //left leads to dead end。 Try going straight
    if (TraverseMaze(intsec[CurrentPos].forward))
      { cout << CurrentPos << "   "; return 1; }
    else              //left, straight lead to dead end。 Try going right
    if (TraverseMaze(intsec[CurrentPos].right))
      { cout << CurrentPos << "   "; return 1;  }
  }
  return 0;   // at a dead end
}
```

【问题描述】在$n \times n$的方格棋盘上，放置n个皇后，要求每个皇后不同行、不同列、不同左右对角线。如下图所示是6皇后问题的一个解。



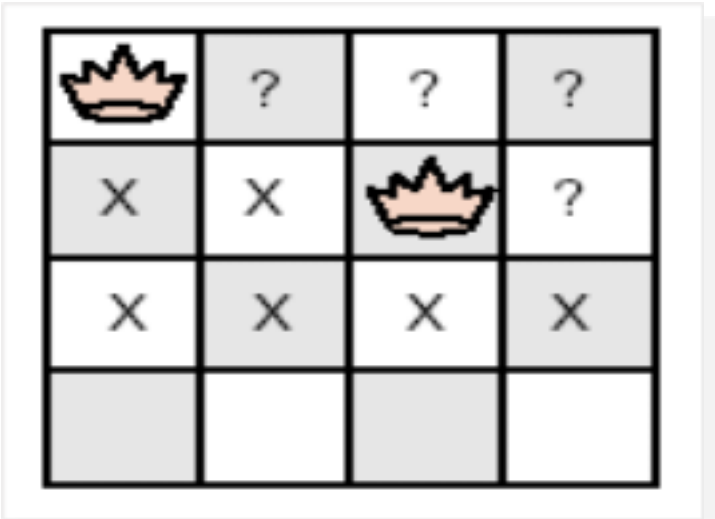
$q[1..6]=\{2，4，6，1，3，5\}$

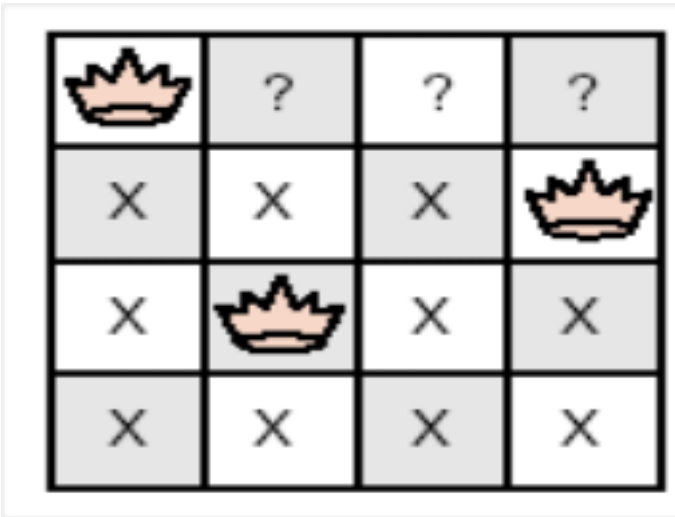
Example

Four queens

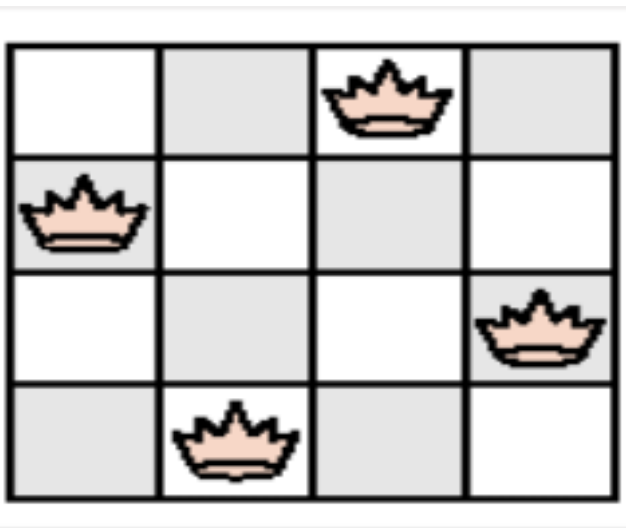
Dead end

Solution



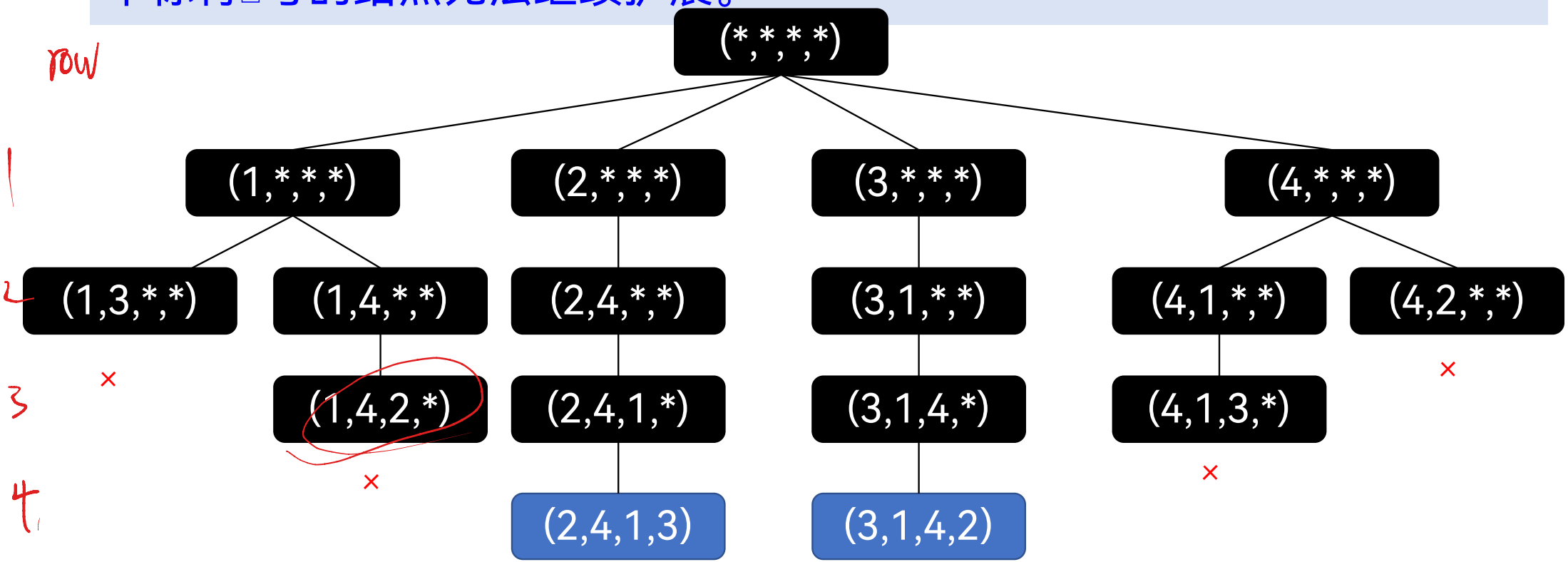






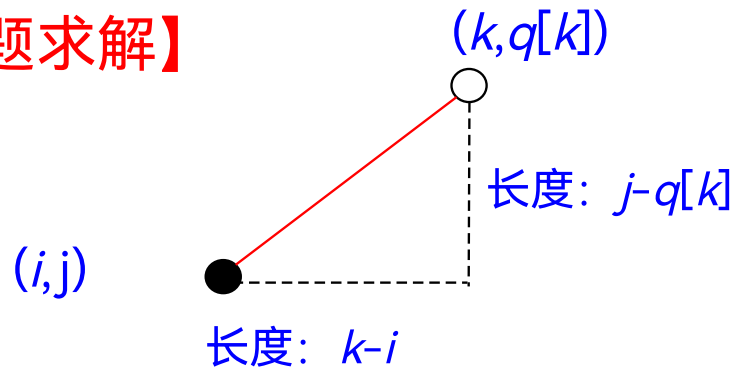
3.3 求解n皇后问题

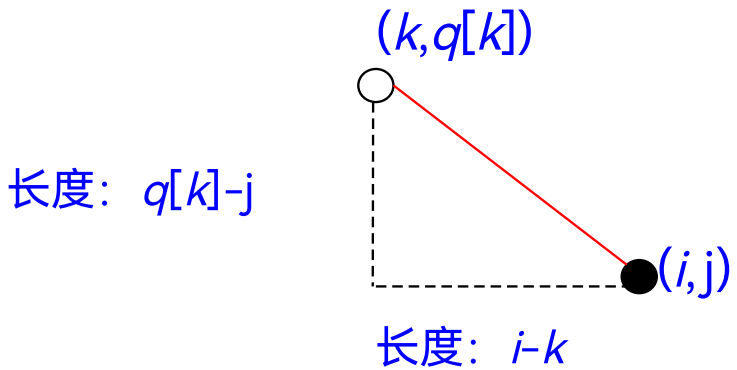
下图是四皇后问题的搜索空间，图中每个状态由当前放置的皇后的行列号构成。它给出了四皇后问题的全部搜索过程，只有18个结点，其中标有□号的结点无法继续扩展。



3.3 求解n皇后问题

【问题求解】





对于(*i*，*j*)位置上的皇后，是否与已放好的皇后(*k*，*q*[*k*])（1≤*k*≤*i*-1）有冲突呢?

- 显然它们不同列，若同列则有：*q*[*k*]==*j*;
- 对角线有两条，若它们在任一条对角线上，则构成一个等边直角三角形，即|*q*[*k*]-*j*|==|i-*k*|。

(*q*[*k*]==*j*) || (abs(*q*[*k*]-*j*)==abs(*i*-*k*))

3.3 求解n皇后问题

设queen(*i*，*n*)是在1 ~ *i*-1行上已经放好了*i*-1个皇后，用于在*i* ~ *n*行放置*n*-*i*+1个皇后，则queen(*i*+1，*n*)表示在1 ~ *i*行上已经放好了*i*个皇后，用于在*i*+1 ~ *n*行放置*n*-*i*个皇后。

queen(*i*+1，*n*)比queen(*i*，*n*)少放置一个皇后。所以queen(*i*+1，*n*)是“小问题”，queen(*i*，*n*)是“大问题”。

queen(*i*，*n*) ≡ *n*个皇后放置完毕，输出一个解　　若*k*>*n*

queen(*i*，*n*) ≡ 在第*i*行的合适的位置（*i*,*j*）　其他情况

　　在其上放置一个皇后;

　　queen(*i*+1，*n*);

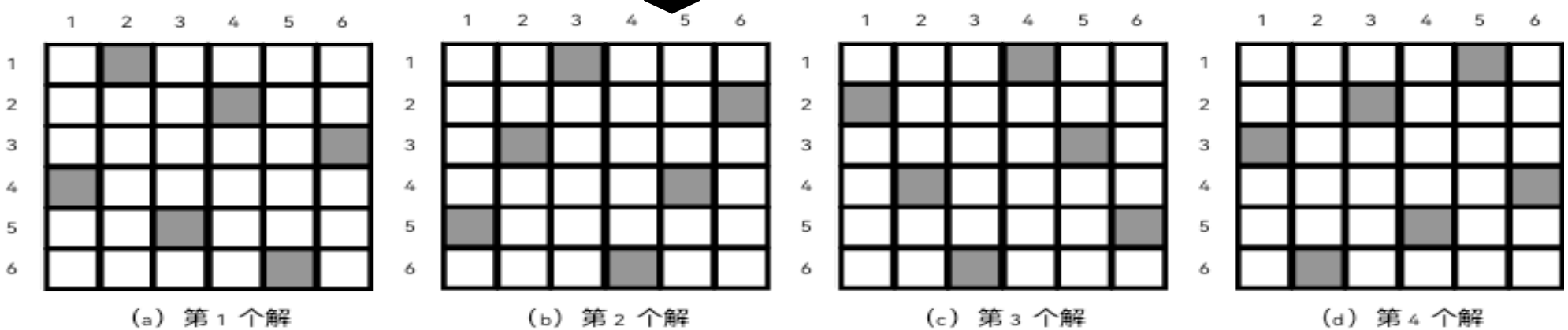
```
bool place(int i,int j)                    //测试(i,j)位置能否摆放皇后
{   if (i==1) return true;                                  //第一个皇后总是可以放置
    int k=1;
    while (k<i)                                             //k=1 ~ i-1是已放置了皇后的行
    {           if ((q[k]==j) || (abs(q[k]-j)==abs(i-k)))
                    return false;
                k++;
    }
    return true;
}

void queen(int i,int n)                                     //放置1 ~ i的皇后
{   if (i>n)
                dispasolution(n);                           //所有皇后放置结束
    else
    {           for (int j=1;j<=n;j++)  //在第i行上试探每一个列j
                    if (place(i,j))                         //在第i行上找到一个合适位置(i,j)
                    {           q[i]=j;
                                queen(i+1,n);
                    }
    }
}
```

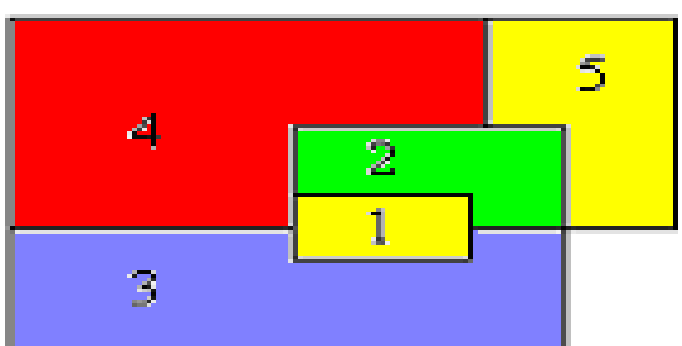
3.3 求解n皇后问题

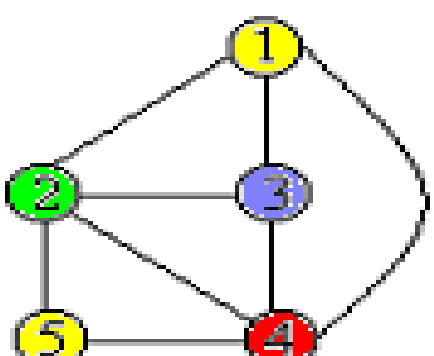
本程序一次执行结果如下：

皇后问题(n<20) n=6↙
6皇后问题求解如下：
第1个解：(1，2) (2，4) (3，6) (4，1) (5，3) (6，5)
第2个解：(1，3) (2，6) (3，2) (4，5) (5，1) (6，4)
第3个解：(1，4) (2，1) (3，5) (4，2) (5，6) (6，3)
第4个解：(1，5) (2，3) (3，1) (4，6) (5，4) (6，2)



给定无向连通图G和m种不同的颜色。用这些颜色为图G的各顶点着色，每个顶点着一种颜色。如果这个图不是m可着色的就给出否定回答，如果是m可着色的就找出有多少种着色方法。这个问题是图的m可着色判定问题。





3.4 图的m着色问题

【问题描述】给定无向连通图G和m种不同的颜色。用这些颜色为图G的各顶点着色，每个顶点着一种颜色。如果有一种着色法使G中每条边的两个顶点着不同颜色，则称这个图是m可着色的。图的m着色问题是对于给定图G和m种颜色，找出所有不同的着色法。

【输入格式】第1行有3个正整数n、k和m，表示给定的图G有n个顶点和k条边，m种颜色。顶点编号为1，2，…，n。接下来的k行中，每行有两个正整数u、v，表示图G的一条边（u，v）。

【输出格式】程序运行结束时，将计算出的不同的着色方案数输出。如果不能着色，程序输出-1。

【输入样例】

5 8 4

1 2

1 3

1 4

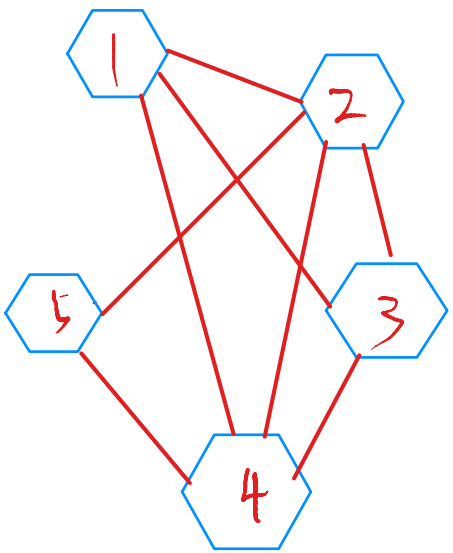
2 3

2 4

2 5

3 4

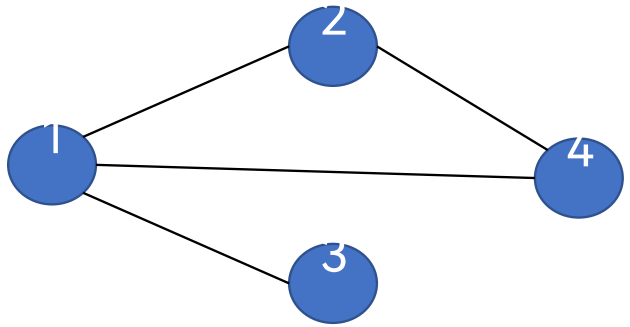
4 5



【输出样例】

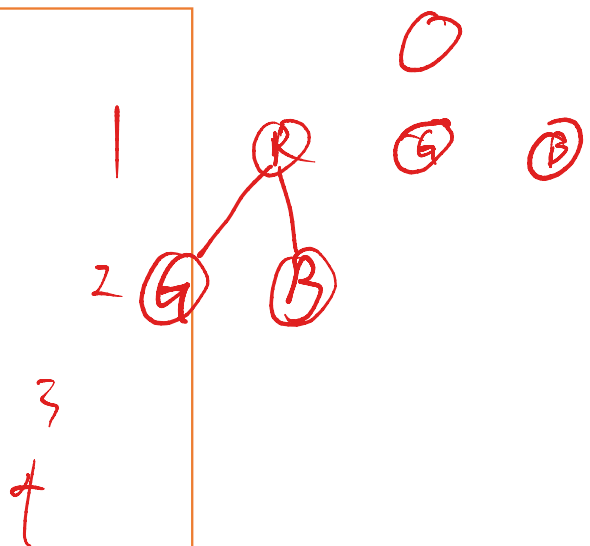
48

3.4 图的m着色问题



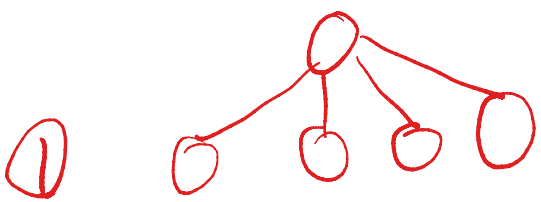
n=4，k=4，m=3，其着色方案有12个。

第1个着色方案：1 2 2 3
第2个着色方案：1 2 3 2
第3个着色方案：1 3 2 3
第4个着色方案：1 3 3 2
第5个着色方案：2 1 1 3
第6个着色方案：2 1 3 1
第7个着色方案：2 3 1 3
第8个着色方案：2 3 3 1
第9个着色方案：3 1 1 2
第10个着色方案：3 1 2 1
第11个着色方案：3 2 1 2
第12个着色方案：3 2 2 1



【问题求解】对于图*G*，采用邻接矩阵*a*存储，根据求解问题需要，这里*a*为一个二维数组（下标0不用），当顶点*i*与顶点*j*有边时，置*a*[*i*][*j*]=1，其他情况置*a*[*i*][*j*]=0。

图中的顶点编号为1 ~ *n*，着色编号为1 ~ *m*。对于图G中的每一个顶点，可能的着色为1 ~ *m*，所以对应的解空间是一棵*m*叉树，高度为*n*，层次*i*从1开始。



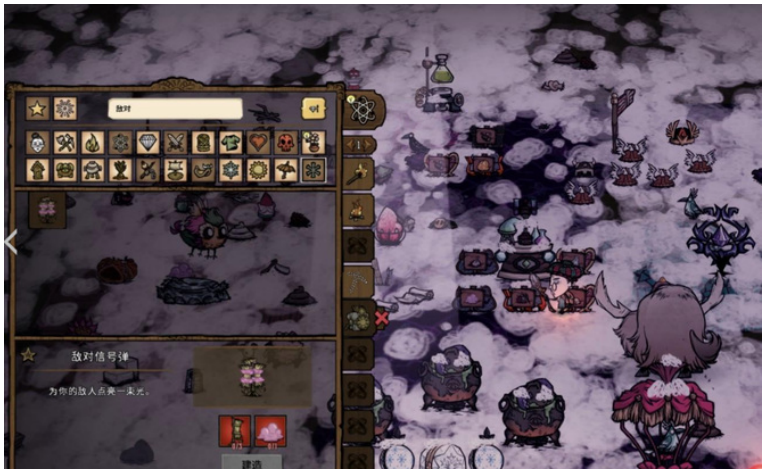
```
bool Same(int i)   //判断顶点i是否与相邻顶点存在相同的着色
{  for (int j=1;j<=n;j++)
     if (a[i][j]==1 && x[i]==x[j])
        return false;
   return true;
}

void dfs(int i)                  //求解图的m着色问题
{  if (i>n)                      //达到叶子结点
      count++;                              //着色方案数增1
   else
   {  for (int j=1;j<=m;j++)     //试探每一种着色
      {  x[i]=j;                 //试探着色j
         if (Same(i))            //可以着色j，进入下一个顶点着色
            dfs(i+1);
         x[i]=0;                 //回溯
      }
   }
}
```

```
int n,k,m;
int a[max][max];
int count=0;
int x[max];
int main()
{   memset(a, 0, sizeof(a));
    memset(x,0,sizeof(x));
    int x,y;
    scanf(“%d%d%d”,&n, &m, &k);
    for(int j=1; j<=k; j++)
    { scanf(%d%d”, &x, &y);
       a[x][y]=1;
       a[y][x]=1;}
     dfs(1);
     if(count>0)
       printf(“%d\n”, count);
     else
       printf(“-1\n”);
     return 0;
}
```

【算法分析】该算法中每个顶点试探1～m种着色，共n个顶点，对应解空间树是一棵m叉树（子集树），算法的时间复杂度为$O(m^n)$。

3.5 0-1背包问题

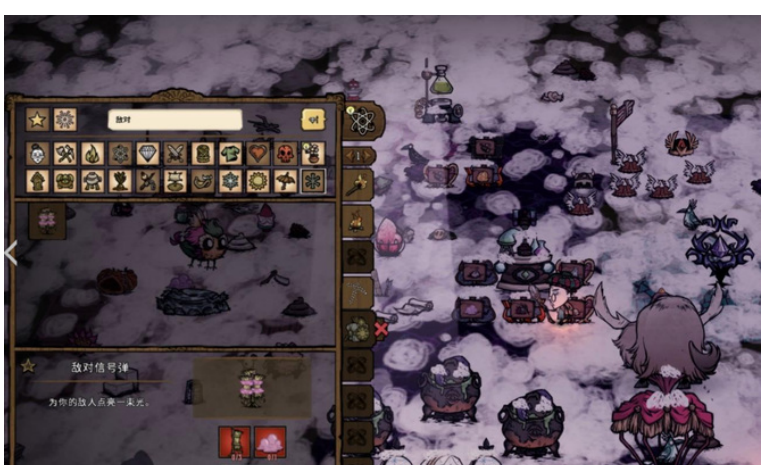






3.5 0-1背包问题

在$0/1$背包问题中，需对容量为c的背包进行装载。从n个物品中选取装入背包的物品，每件物品i的重量为w_i，价值为p_i。对于可行的背包装载，背包中物品的总重量不能超过背包的容量，最佳装载是指所装入的物品价值最高。







用回溯算法搜索子集树的一般模式

```
void search(int m)
{
        if(m>=n)          //递归结束条件
         output();      //相应的处理(输出结果)
        else
        {
         a[m]=0;       //设置状态：0表示不要该物品
         search(m+1);  //递归搜索：继续确定下一个物品
         a[m]=1;       //设置状态：1表示要该物品
         search(m+1);  //递归搜索：继续确定下一个物品
        }
}
```

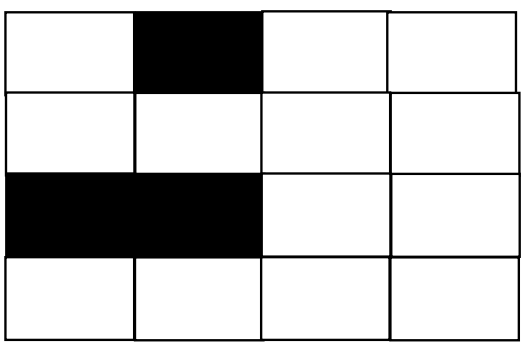
```
int main()
{
        readdata();      //读入数据
        search(0);       //递归搜索
        printresult();
}
```

```
void search(int m)
{
        if(m>=n)
                checkmax();  //检查当前解是否是可行解，若是则把它的价值与
  max比较
        else
        {
                a[m]=0;      //不选第m件物品
                search(m+1); //递归搜索下一件物品
                a[m]=1;      //选第m件物品
                search(m+1); //递归搜索下一件物品
        }
}
```

```
void checkmax()
{
  int i, weight=0, value=0;
  for(i=0;i<n;i++)
  {
        if(a[i]==1)              //如果选取了该物品
        {
              weight = weight + w[i]; //累加重量
              value = value + v[i];   //累加价值
        }
  }
  if(weight<=c)                  //若为可行解
        if(value>max)              //且价值大于max
              max=value;           //替换max
}
```

3.6 堡垒问题

如图城堡是一个4×4的方格，为了保卫城堡，现需要在某些格子里修建一些堡垒。城堡中的某些格子是墙，其余格子都是空格，堡垒只能建在空格里，每个堡垒都可以向上下左右四个方向射击，如果两个堡垒在同一行或同一列，且中间没有墙相隔，则两个堡垒都会把对方打掉。问对于给定的一种状态，最多能够修建几个堡垒。



```
int main()
{
        readdata();      //读入数据
        search(0);       //递归搜索
        printresult();
}
```

```
void search(int m)
{
        if(m>=16)
                checkmax();
        else
        {
                a[m]=0;
                search(m+1);
                if(canplace(m))
                {
                        a[m]=1;
         map[m/4][m%4]=2;
                        search(m+1);
                }
        }
}
```

```
void checkmax()
{
        int i,value=0;
        for(i=0;i<16;i++)
        {
                if(a[i]==1)
                {
                        value = value + 1;
                }
        }
        if(value>max)
                max=value;       //替换max
}
```

```
int canplace(int m)
{  int i,j,row,col;
    row=m/4;
    col=m%4;
   if(map[row][col]!=0) {
          return 0;
   }
   else{
          for(i=row;i>=0;i--) {//从当前行沿着同一列返回检查
                if(map[i][col]==2) {
                      return 0;
                }
                else if(map[i][col]==1) {
                      break;
                }
          }
```

```
            for(j=col;j>=0;j--) {
                if(map[row][j]==2) {
                    return 0;
                }
                else if(map[row][j]==1) {
                    break;
                }
            }
            return 1;
        }
}
```

有两艘船，载重量分别是c1、 c2，n个集装箱，重量是wi (i=1…n)，且所有集装箱的总重量不超过c1+c2。确定是否有可能将所有集装箱全部装入两艘船。

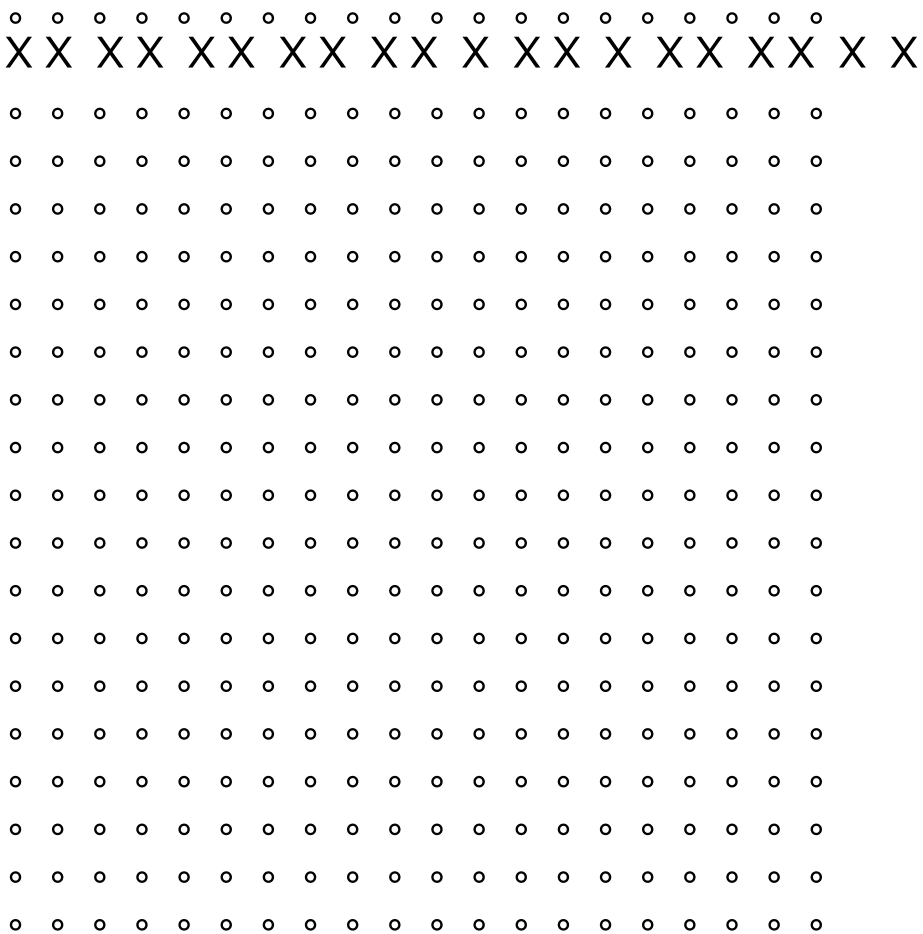
```
int search(int m)
{
        int x,y;
    if(m>n)
     checkmax();
        else
        {       a[m]=0;
                y=search(m+1);
        if(cw+w[m]<=c1)
                {       a[m]=1;
                        cw=cw+w[m];
                        x=search(m+1);
                }
        }
}
```

3.8 迷宫问题

- 给一个20×20的迷宫、起点坐标和终点坐标，问从起点是否能到达终点。
- 输入数据：’。’表示空格；’X’表示墙。

3.8 迷宫问题

0 0 19 19



3.8 迷宫问题

0 0 19 19

```
o o o o o o o o o o o o o o o o o o o o
X XX X X X X X X X X X X X X XXX 。
o o o o o o o o o o o o o o o o o o o o
。X X X X X X X X X X X X X X X XXX
o o o o o o o o o o o o o o o o o o o o
X X X X X X X X X X X X X X X XXX 。
o o o o o o o o o o o o o o o o o o o o
。X X X X X X X X XX X X X X X X XX
o o o o o o o o o o o o o o o o o o o o
X X X X X X X X XX X X XX X X X 。
o o o o o o o o o o o o o o o o o o o o
。X X X X X X X XX X X X XX X X X XX
o o o o o o o o o o o o o o o o o o o o
X X X X X X X XX X X X X X X XX X 。
X X X X X X X XX X X X X X X XX X 。
X X X X X X X XX X X X X X X XX X 。
X X X X X X X XX X X X X X X XX X 。
o o o o o o o o o o o o o o o o o o o o
。X X X X X X XX X X X X X XX X X X X
o o o o o o o o o o o o o o o o o o o o
```

```
void search(int,int);
int canplace(int,int);
void readdata();          //读入数据
void printresult();       //打印结果
int a[20][20];            //a数组存放迷宫
int s,t;//起点与终点的位置
```

```
int main()
{
        int row, col;
        readdata();
        row=s/20;
        col=s%20;
        search(row,col);        //递归搜索
}
```

```
int canplace(int row, int col)
{
  if(row>=0&&row<20&&col>=0&&col<20&&a[row][col]==0)
        return 1;
  else
        return 0;
}
```

```
void search(int row, int col)
{
        if (row*20+col==t)
      printresult();
   int r,c;
        a[row][col]=1;
        r=row;              //左
        c=col-1;
        if(canplace(r,c))   //判断(r,c)位置是否已经走过
                search(r,c);   //递归搜索(r,c)
        r=row+1;            //下
        c=col;
        if(canplace(r,c))   //判断(r,c)位置是否已经走过
                search(r,c);   //递归搜索(r,c)
```

```
    r=row;              //右
        c=col+1;
        if(canplace(r,c))   //判断(r,c)位置是否已经走过
                search(r,c);    //递归搜索(r,c)
        r=row-1;            //上
        c=col;
        if(canplace(r,c))   //判断(r,c)位置是否已经走过
                search(r,c);        //递归搜索(r,c)
}
```

翻硬币问题

把硬币摆放成32×9的矩阵，你可以随意翻转矩阵中的某些行和某些列，问正面朝上的硬币最多有多少枚？

3.10 排列问题

设R={r1,r2,...,rn}是要进行排列的n个元素，Ri=R-{ri}。集合x中元素的全排列记为Perm(X)。(ri)Perm(X)表示在全排列Perm(X)的每一个排列前加上前缀ri得到的排列。R的全排列可归纳如下：

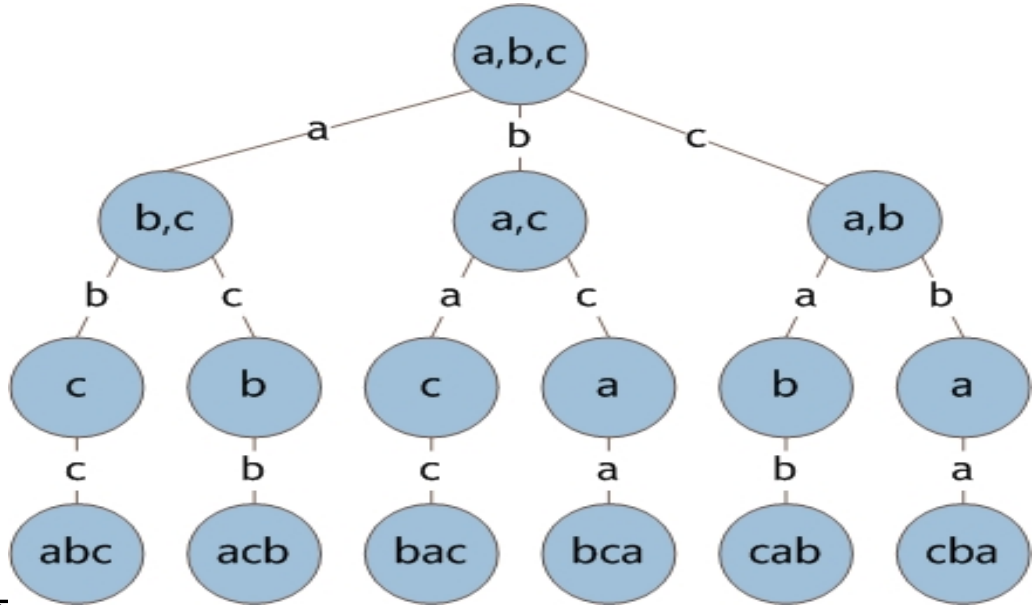
当n=1时，Perm(R)=(r)，其中r是集合中唯一的元素；

当n>1时，Perm(R)由(r1)Perm(R1),(r2)Perm(R2),(r3)Perm(R3).....(rn)Perm(Rn)构成。

遍历排列树需要O(n!)计算时间

```
void backtrack (int t)
{
  if (t>n) output(x);
  else
            for (int i=t;i<=n;i++) {
       swap(x[t], x[i]);
       if (legal(t)) backtrack(t+1);
       swap(x[t], x[i]);
     }
}
```

```
void Perm(Type list[],int k,int m)
{
    //只剩下一个元素
    if(k == m){
        for(int i=0; i<=m; i++)
        {
            cout<<list[i]<<" ";
        }
        cout<<endl;
    }
    else
    {
        //将list[k:m}中的每一个元素分别与list[k]中的元素交换
        //然后递归计算list[k+1:m]的全排列，将计算结果作为list[0:k]后缀
        for(int i=k; i<=m;i++){
            Swap(list[k],list[i]);
            Perm(list,k+1,m);
            Swap(list[k],list[i]);
        }
    }
}
```



3.11 素数环问题

- 把从1到20这20个数摆成一个环，要求相邻的两个数的和是一个素数。
- 分析：用回溯算法，考察所有可能的排列。

```
int main()
{
        int b[12], a[21];
        init();
        search(2);          //递归搜索
}
```

```
int isprime(int num)
{
        int i,k;
        k=sqrt(num);
        for(i=2;i<=k;i++)
                if(num%i==0)
                        return(0);
        return(1);
}
```

```
void getb()
{
        int i,k=0,j,flag;
        for(i=2;i<=39;i++)
        {       flag=1;
                for(j=2;j<=sqrt(i);j++)
                        if(i%j==0)
                        {
                                flag=0;break;
                        }
                if(flag==1)
                        b[k++]=i;
        }
}
```

```
int isprime(int num)
{
        int i;
        for(i=0;i<12;i++)
                if(num==b[i]) return 1;
    return 0;
}
```

```
void init()
{
        int i;
        for(i=0;i<21;i++)
                a[i]=i;
        getb();
}
```

```
void search(int m)
{
  int i;
  if(m>20) //当已经搜索到叶结点时
  {
            if(isprime(a[1]+a[20]))    //如果a[1]+a[20]也是素数
                  printresult();       //输出当前解
            return;
  }
```

```
else
{
        for(i=m;i<=20;i++)      //(排列树)
        {
                swap(m,i);          //交换a[m]和a[i]
                if(isprime(a[m-1]+a[m]))        //判断a[m-1]+a[m]是否是素数
                        search(m+1);    //递归搜索下一个位置
                swap(m,i);          //把a[m]和a[i]换回来
        }
}
}
```

问题描述

某售货员要到若干城市去推销商品，已知各城市之间的路程（旅费），他要选定一条从驻地出发，经过每个城市一遍，最后回到驻地的路线，使总的路程（总旅费）最小。

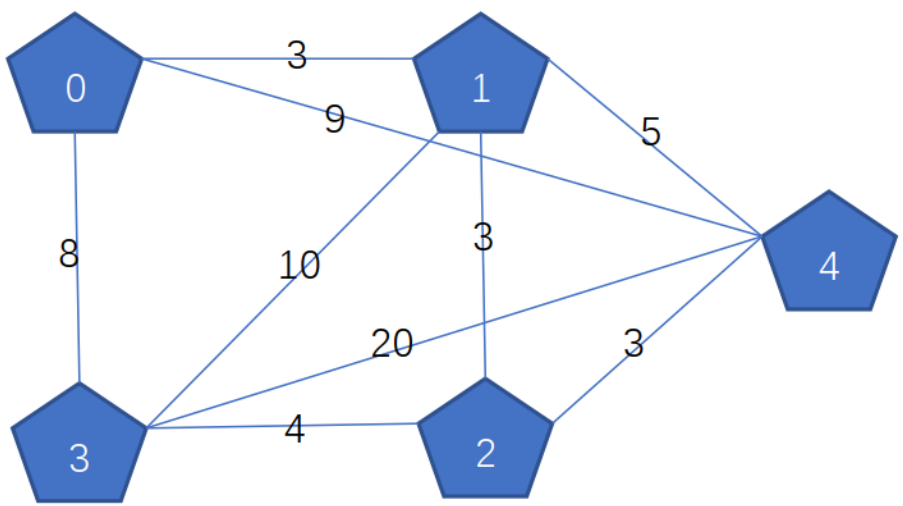
问题分析

旅行售货员问题的解空间是一棵排列树。对于排列树的回溯法与生成1,2,……n的所有排列的递归算法Perm类似。开始时x=[1,2,……n],则相应的排列树有x[1:n]的所有排列构成。

在递归算法Backtrack中，当i=n时，当前扩展节点是排列树的叶节点的父节点。此时算法***检测图G是否存在一条从顶点x[n-1]到顶点x[n]的边和一条从顶点x[n]到顶点1的边***。如果这两条边都存在，则找到一条旅行员售货回路。此时，算法还需要判断这条回路的费用是否优于已找到的当前最优回流的费用bestc。如果是，则必须更新当前最优值bestc和当前最优解bestx。

当i<n时，当前扩展节点位于排列树的第i-1层。***图G中存在从顶点x[i-1]到顶点x[i]的边时，x[1:i]构成图G的一条路径，且当x[1:i]的费用小于当前最优值时算法进入树的第i层***，否则将剪去相应的子树。

3.12 旅行售货员问题


0
1
2
3
4
3
9
5
8
10
3
20
3
4


总长：15733.4公里
起点
244.4公里
516.1公里
13793.7公里
14678.3公里
14567.9公里
14304.3公里
1311.4公里
1683.0公里
3223.8公里
3493.9公里
3660.4公里
乌鲁木齐
11277.7公里
13256.8公里
12913.6公里
2115.4公里
8423.0公里
8168.2公里
9671.3公里
2763.4公里
5234.2公里
7833.6公里
7403.8公里
4500.0公里
4249.2公里
6779.6公里
5802.6公里
5998.4公里
6408.2公里

```
private static void backtrack(int i)
  {
    if (i == n) {
      if (a[x[n - 1]][x[n]] < Float.MAX_VALUE && a[x[n]][1] < Float.MAX_VALUE &&
       (bestc == Float.MAX_VALUE || cc+a[x[n - 1]][x[n]]+a[x[n]][1]<bestc)) {
        for (int j = 1; j <= n; j++)  bestx[j] = x[j];
        bestc = cc+a[x[n - 1]][x[n]]+a[x[n]][1];
      }
    } else {
      for (int j = i; j <= n; j++)
        // 是否可进入x[j]子树?
        if (a[x[i - 1]][x[j]] < Float.MAX_VALUE &&
        (bestc == Float.MAX_VALUE ||  cc+a[x[i - 1]][x[j]]<bestc)) {// 搜索子树
          MyMath.swap(x, i, j);
          cc+=a[x[i - 1]][x[i]];
          backtrack(i + 1);
          cc-=a[x[i - 1]][x[i]];
          MyMath.swap(x, i, j);
        }
    }
```

复杂度分析

算法backtrack在最坏情况下可能需要更新当前最优解O((n-1)!)次，每次更新bestx需计算时间O(n)，从而整个算法的计算时间复杂性为O(n!)。

3.13 批处理作业调度问题

给定n个作业的集合{$J_1,J_2,\ldots,J_n$}。每个作业必须先由机器1处理，然后由机器2处理。作业J_i需要机器j的处理时间为t_{ji}。对于一个确定的作业调度，设F_{ji}是作业i在机器j上完成处理的时间。所有作业在机器2上完成处理的时间和称为该作业调度的完成时间和。

批处理作业调度问题要求对于给定的n个作业，制定最佳作业调度方案，使其完成时间和 t_{ji} 达到最小。

t_{ji}	机器1	机器2
作业1	2	1
作业2	3	1
作业3	2	3

这3个作业的6种可能的调度方案是1,2,3；1,3,2；2,1,3；2,3,1；3,1,2；3,2,1；它们所相应的完成时间和分别是19，18，20，21，19，19。易见，最佳调度方案是1,3,2，其完成时间和为18。

```
private static void backtrack(int i)
 {
     if (i > n) {
      for (int j = 1; j <= n; j++) bestx[j] = x[j];
      bestf = f;
     }  else
      for (int j = i; j <= n; j++) {
        f1+=m[x[j]][1];
        f2[i]=((f2[i-1]>f1)?f2[i-1]:f1)+m[x[j]][2];
        f+=f2[i];
        if (f < bestf) {//如果比当前最短时间短就生成下一步
          MyMath.swap(x,i,j);
          backtrack(i+1);
          MyMath.swap(x,i,j);
          }
        f1-=m[x[j]][1];//否则就不生成了
        f-=f2[i];
        }
  }
```

```
public class FlowShop
     static int  n,      // 作业数
                 f1,     // 机器1完成处理时间
                 f,      // 完成时间和
                 bestf;  // 当前最优值
     static int [][] m;   // 各作业所需的处理时间
     static int [] x;     // 当前作业调度
     static int [] bestx; // 当前最优作业调度
     static int [] f2;    // 机器2完成处理时间
```

作业与PROJECT与知识点、能力映射表

作业次数	考核知识点	Poject	考核能力点	备注
第一次作业	算法运算开销的估算，掌握大O，大Ω，大θ表示法	Poject1：给定代码的算法复杂度大O，大Ω，大θ表示。	自主学习能力； 系统分析与设计能力；	
第二次作业	递归分治法程序设计思路	Poject2：编程实现二分查找和快速排序	实践编程能力； 结果分析能力；	
第三次作业	回溯法程序设计框架	Poject3：编程实现0-1背包问题，装载问题	对已有知识通过设计实验方案，进行分析验证的能力；	